Simulador Mega-Sena (C#)

Regras definidas até agora

1. Mega-Sena: 6 números únicos de 1 a 60.
2. Excluir do globo os números dos **últimos 3 sorteios** (até 18 números; menos se houver repetidos).
3. Cada jogo pode ter **no máximo 1 par consecutivo** (ex.: 10-11). Trio (10-11-12) conta como 2 pares, então é rejeitado.
4. Rejeição simples: sorteia, confere a regra, repete se não passar.

Quantos jogos são possíveis?

Usando os últimos 3 sorteios de exemplo (18 números distintos excluídos, sobram 42):

Etapa	**Combinações**
Mega-Sena completa C(60,6)	50.063.860
Após excluir os 18 números C(42,6)	5.245.786
Após limitar a no máximo 1 par consecutivo	**4.955.700**

Detalhe por quantidade de pares consecutivos (dentro das 5.245.786):

Pares	**Jogos**
0	3.266.249
1	1.689.451
2	275.186 (rejeitado)
3	14.674 (rejeitado)
4	225 (rejeitado)
5	1 (rejeitado)

- Variante: se trio contar como **uma** sequência só (`ContarBlocos`), sobram **5.061.302** jogos.
- O total muda a cada concurso, pois depende de quais números foram excluídos (e de quão "vizinhos" são os que sobraram). Os valores acima valem para os números de exemplo.
- A regra de sequência corta só ~5,5% das combinações que sobram. A exclusão dos 18 números é o filtro forte.
- Lembrete: os filtros não aumentam a chance de acerto. Cada combinação continua com 1 em 50.063.860.

Código

```
using System.Security.Cryptography;

// Últimos 3 sorteios (exemplos: troque pelos resultados reais)
var ultimos = new List<int[]>
{
    new[] { 4, 11, 23, 37, 48, 59 },   // sorteio -1
    new[] { 2, 15, 26, 33, 41, 52 },   // sorteio -2
    new[] { 7, 19, 28, 35, 44, 60 },   // sorteio -3
};

var sorteio = SortearMegaSena(ultimos, maxSequencias: 1);
Console.WriteLine(string.Join(" - ", sorteio.Select(n => n.ToString("00"))));

static int[] SortearMegaSena(IEnumerable<int[]> ultimosSorteios, int maxSequencia
{
    // 1) Monta o globo sem os números dos últimos sorteios
    var excluidos = ultimosSorteios.SelectMany(s => s).ToHashSet();
    var globo = Enumerable.Range(1, 60)
                          .Where(n => !excluidos.Contains(n))
                          .ToArray();

    // 2) Sorteia até satisfazer a regra de sequências
    while (true)
    {
        var bolas = (int[])globo.Clone();

        // Fisher-Yates parcial: só precisamos de 6
        for (int i = 0; i < 6; i++)
        {
            int j = RandomNumberGenerator.GetInt32(i, bolas.Length);
            (bolas[i], bolas[j]) = (bolas[j], bolas[i]);
```

```
        }

        var sorteio = bolas.Take(6).OrderBy(n => n).ToArray();

        if (ContarSequencias(sorteio) <= maxSequencias)
            return sorteio;
    }
}

// Conta pares vizinhos (10-11 = 1 par; 10-11-12 = 2 pares)
static int ContarSequencias(int[] ordenado)
{
    int pares = 0;
    for (int i = 1; i < ordenado.Length; i++)
        if (ordenado[i] - ordenado[i - 1] == 1)
            pares++;
    return pares;
}

// Variante: trio conta como UMA sequência
static int ContarBlocos(int[] ordenado)
{
    int blocos = 0;
    for (int i = 1; i < ordenado.Length; i++)
        if (ordenado[i] - ordenado[i - 1] == 1 &&
            (i == 1 || ordenado[i - 1] - ordenado[i - 2] != 1))
            blocos++;
    return blocos;
}
```

Evolução das ideias (29/10/2026)

Universo reduzido: 38 números

- Excluir os 18 números dos últimos 3 sorteios **e** os 4 números menos sorteados da história.
- Combinações em 38 números: C(38,6) = **2.760.681** (contra 50.063.860 no jogo completo).
- Cobertura: o sorteio real cai inteiro dentro desses 38 números em apenas ~5,5% dos concursos. Cada jogo continua valendo 1 em 50.063.860.
- Os números menos sorteados não têm menos chance: a diferença de frequência é ruído estatístico (confirmável com teste qui-quadrado).

Índice com ventilador e peso da bolinha

- Ideia: simulador de física (bolas com massa e raio, ventilador com velocidade, colisões, saída por orifício) que gere um índice/sorteio a partir de condições iniciais.
- Na prática vira um gerador pseudoaleatório mais elaborado, sem poder preditivo. Projeto de programação, não de vantagem.

O que é Monte Carlo

Rodar o experimento aleatório milhões de vezes e medir o resultado, em vez de usar fórmula. Exemplos: quantos sorteios o filtro descarta, quantas vezes um jogo faria quadra, quina ou sena.

Simulador de física 2D (versão 1) — pasta `MegaSenaFisica/`

Arquivos: `SimConfig.cs` (parâmetros), `SimuladorFisico.cs` (motor), `Program.cs` (uso), `MegaSenaFisica.csproj`.

Como rodar:

```
dotnet run                      # gera 1 jogo (globo sem últimos 3 sorteios e 4 m
dotnet run -- freq 300 60       # testa uniformidade: 300 sorteios, globo de 60 (
```

O que simula: globo circular giratório, jato de ar (ventilador) com turbulência, gravidade, arrasto, colisões bola-bola e bola-parede, atrito da parede em rotação, saída por uma abertura no topo. Depois de cada bola sair, a abertura fecha e o globo mistura de novo.

Parâmetros (em `SimConfig.cs`):

Parâmetro	Valor	Origem
Massa da bola	66 g	oficial (Caixa)
Diâmetro da bola	50 mm	oficial (Caixa)
Raio do globo 2D	0,45 m	estimado (em 2D precisa de mais área que o globo 3D real)
Rotação do globo	30 rpm	estimado
Jato do ventilador	25 m/s², coluna de 0,20 m	estimado
Turbulência	0,5	estimado

Restituição bola-bola / parede	0,80 / 0,70	estimado

Resultado do primeiro teste (300 sorteios, globo de 60 bolas): qui-quadrado = 56,9, abaixo do limite de ~77,9 (95%, 59 gl). Ou seja, sem viés detectável: as bolas saem de forma uniforme, e o simulador funciona como um gerador aleatório.

Observações:

- Como todas as bolas pesam o mesmo, o peso não diferencia as bolas. O resultado vem de posição inicial, jato e colisões.
- Os números de `ultimosSorteios` e `menosSorteados` no `Program.cs` são exemplos: troque pelos reais.
- Compilado e testado com .NET 8. Sem internet, use `dotnet build --source <pasta vazia>`.

Monte Carlo (comando `dotnet run -- monte 20000000`)

Arquivo `MonteCarlo.cs`. Simula 20 milhões de sorteios reais (sempre nas 60 bolas) e, para cada um, gera uma aposta de cada universo com o filtro de no máximo 1 par consecutivo.

Universo	Jogos rejeitados pelo filtro	Sorteio real cai dentro	Quadras	Quinas
60 (completo)	7,4%	100%	8.505	137
42 (sem últimos 3)	5,5%	10,48%	8.566	104
38 (sem últimos 3 e 4 menos)	4,0%	5,51%	8.585	132

Esperado para qualquer aposta fixa em 20 milhões: ~8.575 quadras e ~129 quinas. Os três universos ficaram nesse valor, então reduzir o universo não melhora a chance de acerto.

Jogos válidos (exatos, com no máximo 1 par consecutivo), com os números de exemplo:

Universo	Combinações	Após o filtro
60	50.063.860	46.383.480
42	5.245.786	4.955.700

38	2.760.681	2.651.467

Correção: numa versão anterior eu disse que cerca de 40% dos sorteios aleatórios têm 2 pares ou mais. O certo é ~7,4% (no globo de 60 bolas), então o filtro de sequências corta pouco.

Animação do globo

Página `globo-megasena.html` (também publicada como artifact): canvas com o globo, o jato de ar e as bolas numeradas, com controles de ventilador, rotação, turbulência e quique, universo (60/42/38) e velocidade. Usa a mesma física do `SimuladorFisico.cs`, para calibrar os parâmetros a olho.

Próximos passos

1. Calibrar os parâmetros do `SimConfig.cs` olhando a animação (e portar os valores escolhidos para o C#).
2. Análise estatística do histórico real (frequência por número e qui-quadrado).
3. Gerar N jogos distintos de uma vez, apostas de 7 a 15 dezenas e leitura dos últimos sorteios de arquivo/API.
4. Usar o simulador físico dentro do Monte Carlo (hoje o Monte Carlo usa sorteio direto, por ser rápido).